

Dispensa Weekly Report

28 September - 2 October 2026

In its very first week, Dispensa grew from an empty folder into a small, working computer program with its own database and a first real feature that restaurants can actually use.

The week in three sentences

This week Aref and Claude wrote the project's design document (a plan for how the system works), got a first version of the program running, connected it to a real database, and built the first complete feature: a list of ingredients that a restaurant can add, view, change and archive.

The roadmap (the week-by-week project plan in ROADMAP.md) splits the first ten weeks into small chunks, and what got built this week already covers work planned for weeks 1, 2 and 3 - so the project is running ahead of schedule.

The one thing behind schedule is bookkeeping, not building: most of the small checkboxes in the roadmap were not ticked off as the work was finished, which makes the plan look less advanced on paper than the project really is (see page 3).

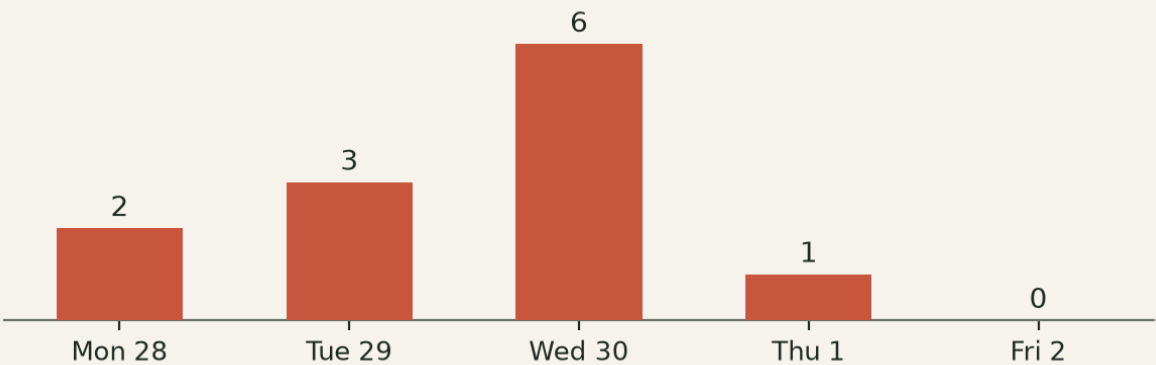
Numbers at a glance



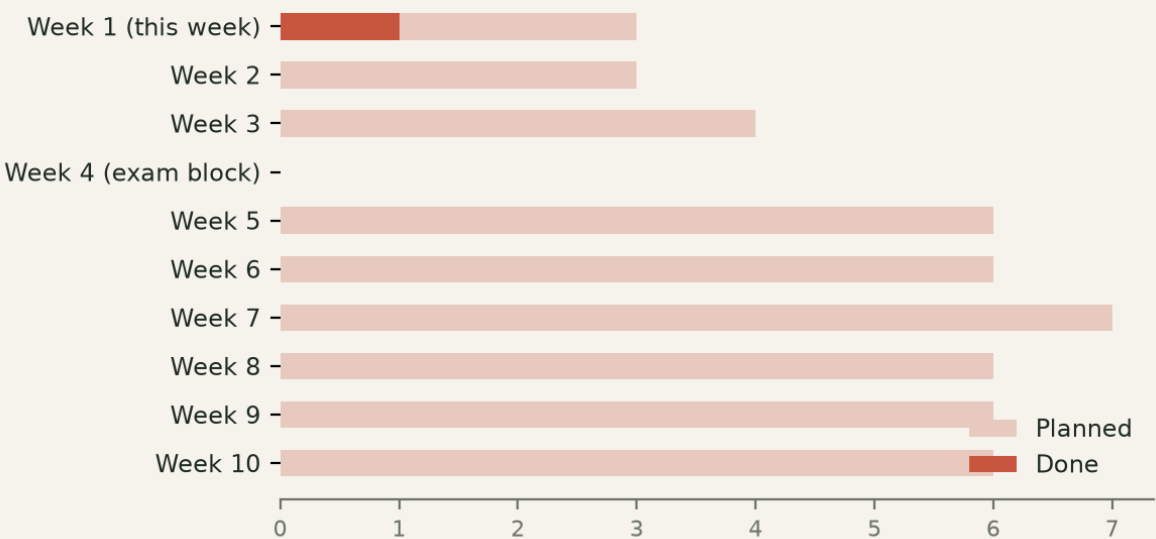
The last card looks low on purpose: it counts only the little checkboxes ticked in the roadmap file itself, not the actual amount of work done. See the note above.

The week in charts

Commits per day (Mon-Fri)



Roadmap progress: planned vs. done, per week



This is the project's first week, so there is only one week of test-count history so far - a chart of tests over time will start appearing from next week's report.

What was built

Each item below is a pull request (PR): a bundled, reviewed change that was merged into the project this week, in the order it happened.

#4 - System design document

Wrote the single reference document for how Dispensa works end to end, and used it to catch a real bug - and four unclear rules - before any code existed.

#5 - Project skeleton

Got the very first, tiny version of the program running, able to answer a simple request over the web.

#6 - Automatic code checks

Set up checks that run automatically before every save and block code with formatting or quality problems, so mistakes are caught immediately.

#7 - Database connection

Connected the program to a real database (PostgreSQL) that keeps all data safe even when the program restarts.

#8 - First data tables

Created the first three data tables (restaurants, users, ingredients) and the first "migration" - a recorded, reversible change to the database's structure.

#9 - Ingredients feature

Built the first complete, end-to-end feature: a restaurant can add, list, view, change and archive its ingredients, each restaurant seeing only its own data, proven by 11 automated tests.

#10 - Work log

Started a running work diary, so decisions and lessons from each session are written down instead of being forgotten - this report reads from it.

Decisions made, and why

All of these come from this week's design document (docs/DESIGN.md, section 10):

Record every change, never store a single "current stock" number	Every number stays explainable; nothing is silently overwritten
Pull recipes & sales into this first release	Without them, the staff role would have almost nothing to do
Only the owner records deliveries	The owner is the one who handles supplier invoices and prices
Staff can't see stock levels or prices	Owner's choice - staff only need to know what dishes are available
Allow stock to go negative, but flag it	A real sale or real waste must never be refused by the system
Deactivate staff accounts, never delete them	Keeps a record of who did what, even after they leave
One shared database for every restaurant	Simple to run; each restaurant's data is kept apart by one shared rule, not a separate database
Login tokens last 8 hours, no auto-renew	Simple to build; matches one work shift
Reading supplier invoices from photos: postponed	Needs file storage and extra safeguards not planned for this release
Simpler (non-concurrent) database access style	Easier to write and test; fast enough for the target of ~100 restaurants

What was learned, and problems solved

- A design review done before writing code caught a real bug while it was still a sentence in a document, not after it was already built into the database.
- Automatic code-quality tools run in their own separate, isolated space - they don't automatically know about the libraries the project uses, and have to be told.
- Checking "does this ingredient belong to this restaurant" in a single question to the database - rather than two separate steps - is what lets a wrong-restaurant request and a not-found request look identical from the outside, which is good for security.
- Testing against a real, freshly-built database schema (not a shortcut) catches a broken database change that a shortcut would silently miss.
- On Windows, the address "localhost" was resolving to the wrong kind of network address and adding about 90 seconds to every database command - fixed by using the exact numeric address instead.
- A generated database change kept failing a formatting check even right after running the formatter - the real cause was an earlier command in the same chain quietly failing first.
- Two pull requests were briefly believed to be merged when they weren't yet - caught by double-checking against GitHub directly instead of trusting the first report.

Still open

- Open issue: #1, "GitHub profile: photo, bio, profile README" - a week-1 task not yet done.
- Open pull requests: none awaiting review at the time of this report.
- Unfinished work: none - the project's main branch is clean, with nothing half-done.
- Housekeeping: the roadmap's checkboxes for weeks 1-3 need to be ticked to match the work that is actually done.

Next week: Week 2 of the roadmap (5-11 Oct)

- Finish the design document's remaining sections (data model, consistency, performance, background jobs, operations) - largely already written this week; mainly needs a check.
- A design review session to challenge the plan and fix anything it misses.
- The project skeleton and first local run - also already in place from this week; worth confirming rather than repeating.

Three questions to check understanding

1. The function that says which restaurant a request belongs to refuses to run at all unless the system is explicitly set to "development" mode. What could go wrong in the live, real-world version of the system if that safety check were removed?

2. When looking up one restaurant's ingredient, the system checks "does this ingredient ID belong to this restaurant ID" in one single question, instead of first finding the ingredient and then checking who it belongs to. What is the security difference between those two approaches?

3. The automated tests build their test database by running the real, same database changes used in production, instead of a quick shortcut that just creates matching tables. What kind of mistake would the shortcut miss that the real process catches?

(Answers are at the very bottom of this page - try answering first.)

Small dictionary

API - A set of web addresses a program exposes so other programs can read or change its data.

Repository (repo) - A project's folder, tracked by git, with its full history of changes.

Merge - Combining a finished piece of work back into the project's main line of code.

Branch - A separate copy of the code to work on something without disturbing the main version.

Migration - A recorded, reversible change to a database's structure, stored like code.

Multi-tenant - One system shared by many customers (restaurants), each seeing only their own data.

Automated test - Code that checks other code automatically, to catch mistakes early.

Backend - The part of a program that runs on a server, out of sight, handling data and rules.

Commit - One saved, labelled snapshot of changes to the code.

Pull request (PR) - A proposed change, shown for review before it is merged.

Database - Organised storage, outside the program itself, that keeps data safe between restarts.

CRUD - The four basic actions on data: Create, Read, Update, Delete.

Ledger - A record where nothing is ever edited, only added to - like a bank statement.

Roadmap - The week-by-week plan for building the project.

Answers to this week's questions

Without that check, the "development" shortcut (which always pretends every request comes from one test restaurant) could run for real, live requests too - meaning every restaurant's request could quietly be treated as the same one test restaurant, breaking data isolation completely.

Checking both IDs together means a request for "this item, but from the wrong restaurant" and a request for "an item that doesn't exist at all" come back looking exactly the same (a plain "not found"). That stops anyone from learning which IDs exist in other restaurants just by poking at the system.

A shortcut that creates matching tables directly would still produce a working-looking test database even if a real migration file were broken or out of date - so a broken, unusable migration could sit in the project while every test kept passing. Running the real migration makes a broken migration fail the tests too.